0011. Container With Most Water

n | Category: Two Pointers | LeetCode: https://leetcode.com/problems/container-

Problem

- Height[i] is a vertical line at x = i. Pick two lines that, with the x-axis, hold the most water. Return that area.
- Area(l, r) = (r - l) min(height[l], height[r]). Example: [1,8,6,2,5,4,8,3,7] → 49.

Intuition

- Start with the widest container (l = 0, r = n-1).
- Moving the taller wall inward can never help: width shrinks and the height is still capped by the shorter wall.
- So always move the shorter wall; that is the only move that might find a taller limit.

Brute Force

- Try all pairs: O(n^2).

Optimal Approach

- L = 0, r = n-1, best = 0.
- Best = max(best, (r-l) min(h[l], h[r])).
- If h[l] < h[r], l++; else r--.

Worked Example

```
height = [1,8,6,2,5,4,8,3,7].

l=0(1) r=8(7) area=8*1=8   move l
l=1(8) r=8(7) area=7*7=49  move r
l=1(8) r=7(3) area=6*3=18  move r
l=1(8) r=6(8) area=5*8=40  move r (tie)
... no later area beats 49 -> 49
```

Java Solution

```java
public class ContainerWithMostWater {
    public int maxArea(int[] height) {
        int l = 0, r = height.length - 1, best = 0;
        while (l < r) {
            best = Math.max(best, (r - l) * Math.min(height[l], height[r]));
            if (height[l] < height[r]) l++;
            else r--;
        }
        return best;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- It is not Trapping Rain Water; only two walls matter, everything between is ignored.